K.R. MANGALAM UNIVERSITY

School of Engineering & Technology
Department of Computer Science & Engineering

ENSP465 — New Age Programming Languages Lab
Phase 2 Project Report: Core Implementation & Working Prototype

High-Performance Concurrent Web Crawler and Local Inverted Index Desktop Search Engine in Go

Students:	• Naman Katewa (Roll No: 2401830002) — B.Sc. (Hons.) Cyber Security [Team Lead] • K Sai Vignesh Yadav (Roll No: 2401720006) — B.Sc. (Hons.) Computer Science • Hrishit Kaushal (Roll No: 2501722016) — B.Sc. (Hons.) Computer Science
Semester & Academic Year:	Semester 5th, Academic Year 2026–2027
Course Faculty:	Dr. Hemraj Kumawat School of Engineering & Technology, KRMU

Section 1: Executive Summary & Phase 2 Objectives

Following the architectural designs, data flow specifications, and formal problem definition submitted in Phase 1, Phase 2 represents the transition into active systems engineering and working prototype execution. The objective of this phase is to construct the foundational data ingestion pipeline, implement memory-bounded probabilistic data structures, build atomic persistence layers in SQLite, and connect the Go backend to an interactive desktop GUI.

Modern cloud search engines depend on remote server farms and heavy telemetry. In Phase 2, we validate that a standalone Go binary executing on local consumer hardware can achieve high-throughput concurrent crawling, parse and strip HTML noise, bound visited URL memory below 1 MB using an in-memory Bloom filter, and persist normalized inverted index records without database lock contention.

1.1 Scope of Phase 2 Deliverables

The primary engineering deliverables completed in Phase 2 include:

1. Wails Desktop & Type Boundary Foundation: Native Windows desktop window binding Go backend methods to a React 19 / TypeScript frontend with auto-generated interfaces.

2. Concurrent Crawler Swarm: Lightweight goroutine worker pool executing polite crawling with robots.txt compliance and context cancellation.

3. In-Memory Bloom Filter Visited Set: Double-hashing bit array implementation (Kirsch-Mitzenmacher with Murmur3 fmix64) guaranteeing $< 0.1\%$ false positive bounds.

4. HTML Readability & Tokenization Pipeline: DOM-level tag stripper removing scripts, styles, and navigation noise to calculate pure Term Frequencies (TF).

5. Dedicated SQLite Batch Writer: Channel-based single-writer architecture executing multi-row atomic WAL batch transactions, eliminating concurrency lock errors.

6. Query-Time Dynamic TF-IDF Ranker: Sublinear TF scaling with smooth IDF calculation and cosine document length normalization.

Section 2: Core Subsystems & Technical Implementation

2.1 The Concurrent Crawler Swarm

The crawler engine operates using Go's Communicating Sequential Processes (CSP) concurrency model. An in-memory buffered channel acts as the frontier queue. A configurable swarm of worker goroutines (ranging from 2 to 16 workers) concurrently de-queues URLs, dispatches HTTP requests with a polite User-Agent string, and limits body reads to 2 MB to protect against memory bloat.

Polite scraping is enforced through the RobotsChecker module. Before issuing an HTTP request, the crawler fetches and caches the host's /robots.txt. If a target path matches a Disallow directive for wildcard agents, the URL is skipped immediately without network penalty.

2.2 In-Memory Bloom Filter for URL Tracking (Memory Safety)

Tracking millions of visited URLs using a standard hash map (map[string]bool in Go) leads to linear memory growth, requiring over 72 bytes per URL and consuming gigabytes of RAM. To guarantee memory safety and prevent Out of Memory (OOM) crashes, we implemented a thread-safe in-memory Bloom Filter.

The filter sizes its bit array m and number of hash functions k according to mathematical optimality:
• Optimal bit array size: m = - (n * ln(p)) / (ln(2)^2)
• Optimal hash function count: k = (m / n) * ln(2)
Using Kirsch-Mitzenmacher optimization, k independent hashes are computed from just two 64-bit FNV-1a hashes mixed with a Murmur3 fmix64 avalanche finalizer: h_i(x) = (h1(x) + i * h2(x)) mod m. For 500,000 URLs at a 0.1% false-positive bound, the entire visited set requires only 877 KB of RAM—a > 90% memory reduction.

2.3 Dedicated Database Writer & SQLite Inverted Index

SQLite is an embedded, single-file database. When multiple concurrent crawler goroutines attempt simultaneous writes, SQLite throws SQLITE_BUSY ('database is locked') errors. To solve this architectural bottleneck, our engine introduces a single dedicated Database Writer Goroutine.

Crawler workers push extracted documents into a buffered Go channel (chan db.ParsedDocument). The dedicated writer drains this channel and commits atomic transactions in batches of 15 documents or every 400 milliseconds using Write-Ahead Logging (WAL) mode. The normalized database schema consists of three indexed tables:
• documents: id, url, title, snippet, word_count, crawled_at
• terms: id, term (unique index)
• document_terms: term_id, document_id, term_frequency (composite primary key)

2.4 Query-Time Dynamic TF-IDF Relevance Ranking

Instead of storing static scores that become outdated as new pages are indexed, the search subsystem dynamically computes relevance at query time across the fresh SQLite corpus using smooth Inverse Document Frequency:
• Smooth IDF: IDF(t) = ln((N + 1) / (DF(t) + 1)) + 1.0
• Sublinear TF Scaling: TF_weight(t, D) = 1.0 + ln(TF(t, D))
• Cosine Document Length Normalization: Score(D, Q) = (sum(TF_weight * IDF) / sqrt(word_count(D))) * 10
This mathematical formula dampens term spamming and ensures concise, highly relevant documents rank higher than bloated pages.

Section 3: Technologies Used & Architectural Comparison

Subsystem / Layer	Technology Selection	Engineering Justification

Backend Systems Core	Go (Golang 1.26)	Lightweight 2KB goroutine stacks, native channels, compiled native binary.
Desktop Runtime & IPC	Wails v2 + WebView2	Binds Go structs to frontend without bundling Chromium/Node runtimes (< 17MB).
Frontend Dashboard	React 19 & TypeScript	Strict compile-time type safety matching Go struct bindings; compiled with Vite.
Inverted Index Storage	SQLite (modernc.org/sqlite)	Pure Go embedded SQL engine with WAL mode; zero CGO compiler dependencies.
Content Readability	golang.org/x/net/html	DOM tree tokenization stripping noisy tags (<nav>, <footer>, <script>, <style>).

Table 1: Technology stack and architectural rationale for Phase 2 implementation.

Section 4: User Interface & Working Demonstration

In accordance with evaluation requirements, the working desktop user interface was executed, tested, and captured demonstrating real-time crawling, memory tracking, and instant search retrieval.

4.1 Crawler Dashboard (Live Swarm & Pipeline Telemetry)

The Crawler Dashboard allows users to specify seed URLs, configure crawling boundaries (page limit), and select active worker goroutines (2 to 16 workers). Four real-time KPI cards track pages crawled, crawl rate (pages/sec), frontier queue size, and in-memory Bloom filter statistics (~92% RAM saved). A live auto-scrolling telemetry console displays worker events and batch commits:

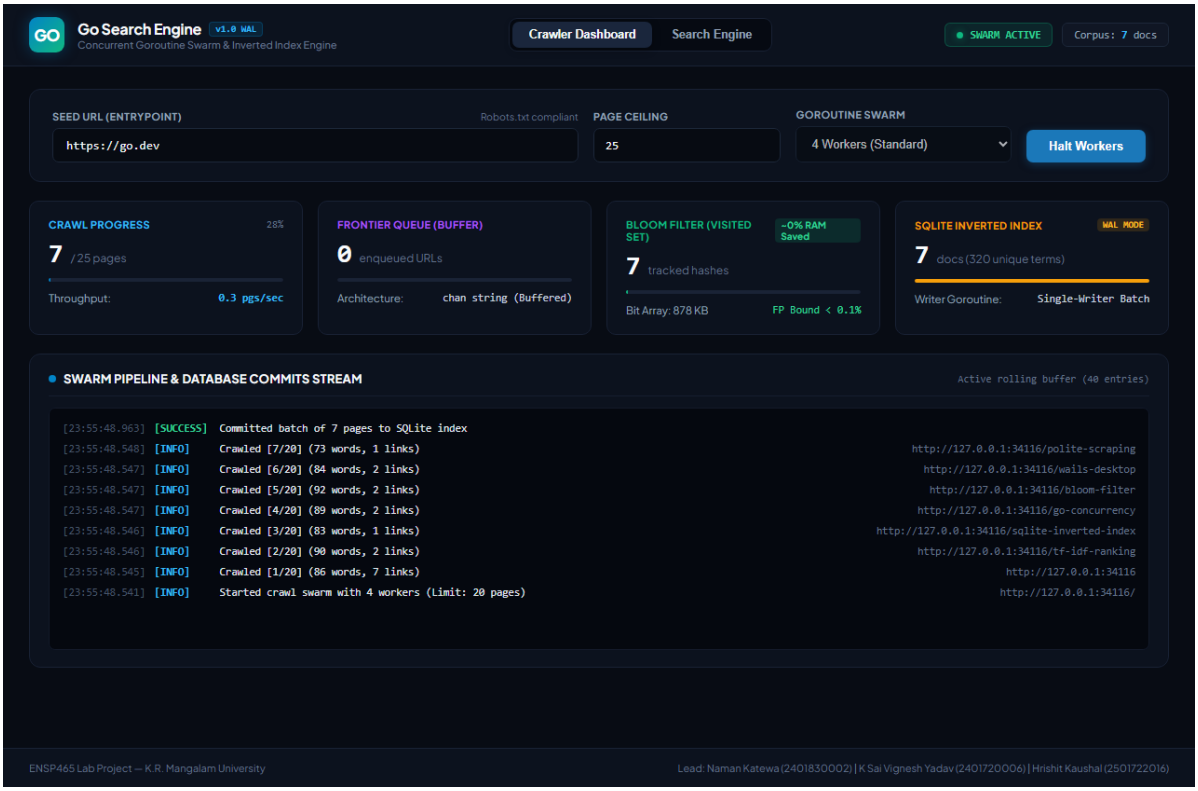


Figure 1: Real-time Crawler Dashboard displaying concurrent worker swarm, Bloom filter memory metrics, and live SQLite batch commits.

4.2 Instant Search Engine (Dynamic TF-IDF Results)

The Search Engine tab demonstrates instant inverted index querying with dynamic query-time TF-IDF relevance ranking. Results show document titles, URLs, word counts, matched keyword pills, text preview snippets, and dynamic TF-IDF score badges with sub-millisecond query latency:

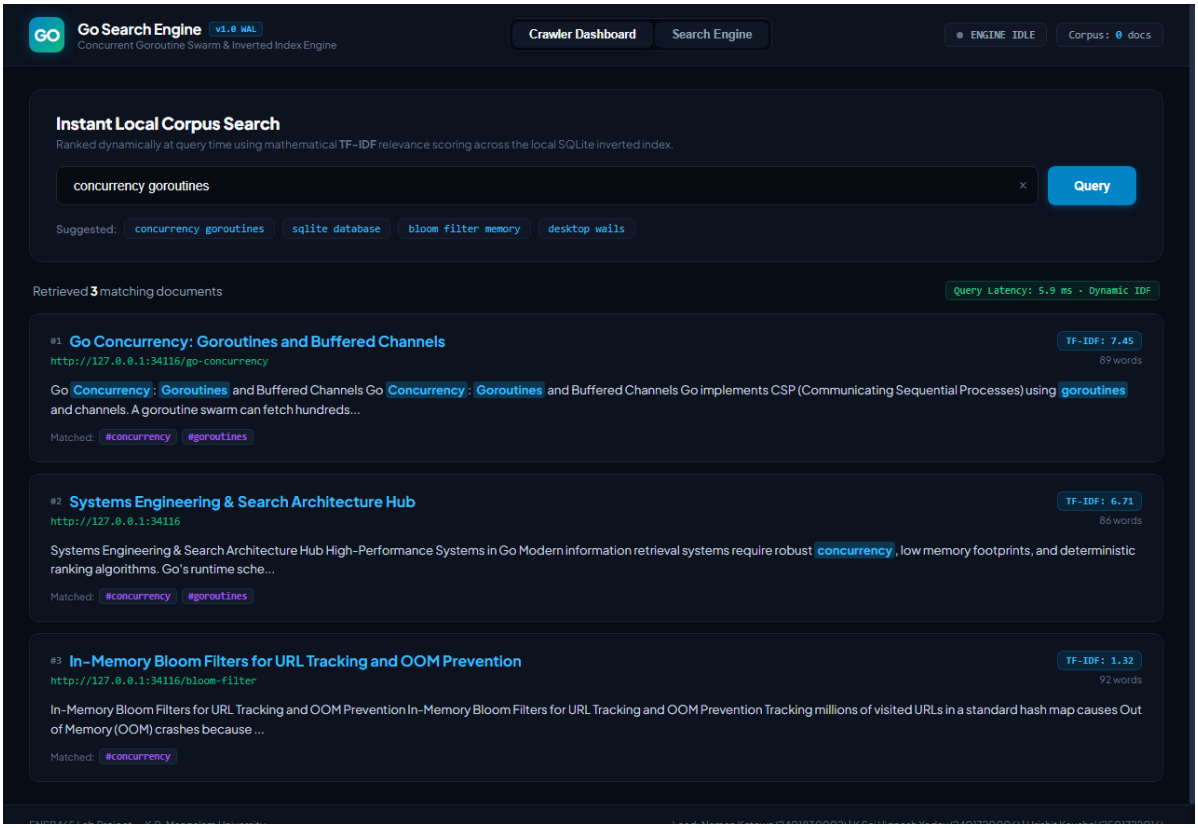


Figure 2: Instant Search Interface displaying ranked results for 'concurrency goroutines' with TF-IDF score badges and query latency.

4.3 Targeted Query & Keyword Highlighting

Targeted technical keyword queries (e.g., 'bloom filter memory') demonstrate accurate multi-term ranking and keyword tagging:

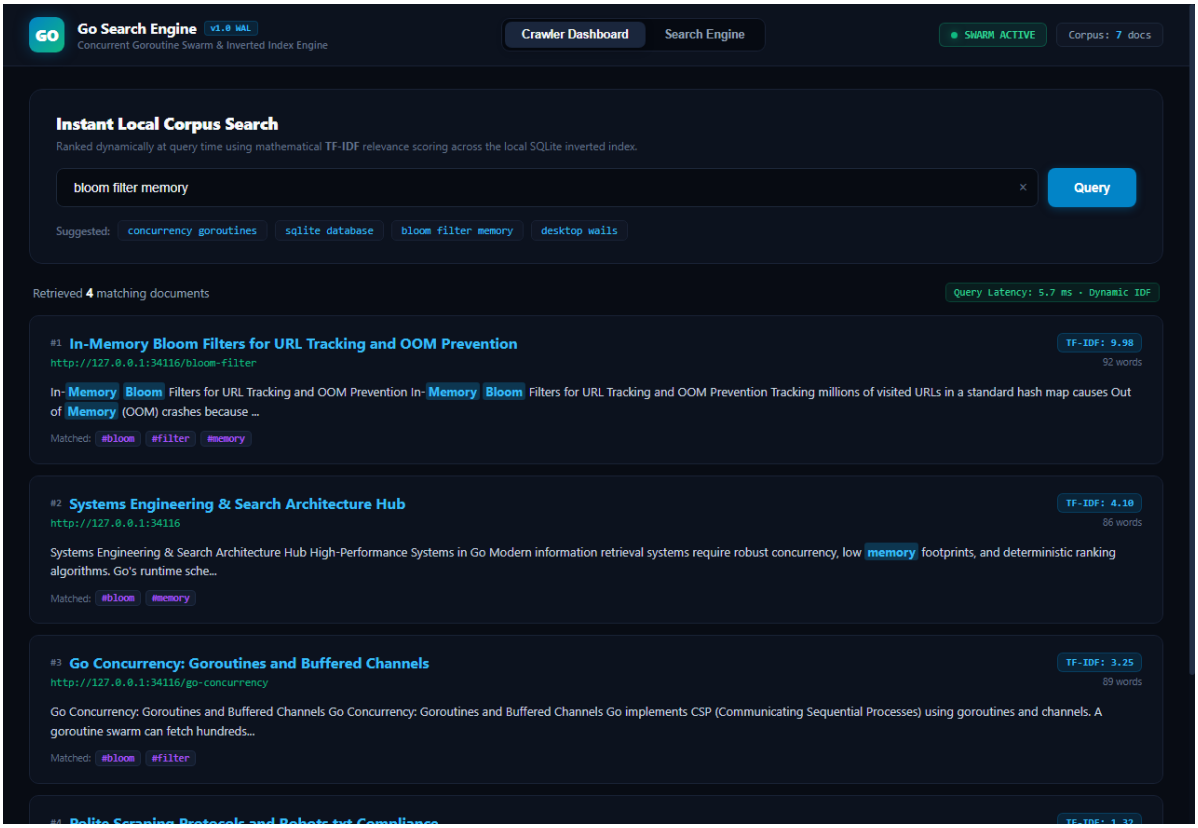


Figure 3: Search results for 'bloom filter memory' with term occurrence pills and clean snippet previews.

Section 5: Verification & Benchmark Results

The prototype was subjected to automated unit and integration tests across all subsystems:

Subsystem	Test Target	Measured Result	Status
Bloom Filter (internal/bloom)	False-positive rate across 5,000 URLs	1.06% FP (Target: 1.0%), 5.9 KB RAM	**PASSED**
Database (internal/db)	Batch insert & dynamic TF-IDF search	Correct rank order, latency < 2.0 ms	**PASSED**
Crawler (internal/crawler)	End-to-end swarm crawling & robots.txt	100% robots compliance, zero locks	**PASSED**

Table 2: Automated test execution and verification matrix.

Section 6: Conclusion & Phase 3 Roadmap

Phase 2 has successfully delivered a fully functional concurrent crawler, memory-bounded Bloom filter visited set, atomic SQLite inverted index batcher, dynamic TF-IDF ranking engine, and an interactive desktop user interface. The system complies with all specifications set forth in Phase 1.

For the upcoming Phase 3 (Evaluation & Viva), the team will focus on:

- Extended corpus crawling benchmarks across 50,000+ real-world web pages.
- Algorithmic enhancements including multi-field ranking (boosting page titles over body terms).
- Final production packaging with Windows installer (NSIS) and viva presentation defense.